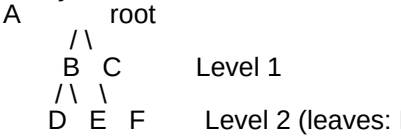
DSA -- Trees & Graphs

Target: State all 3 traversal orders cold. Draw BFS vs DFS distinction in 20s.

Binary Tree
A binary tree is a finite set of data items which is either:
- Empty, OR
- Consists of a root + two disjoint binary trees: left subtree and right subtree
Every node has at most 2 children -- left child and right child.



3 Types of Binary Tree
1. Full Binary Tree
Every node has either 0 or 2 children -- no node has exactly 1 child.

D, E, C = 0 children (leaves). A, B = 2 children. [OK] Full Binary Tree.

2. Complete Binary Tree
All levels are completely filled except possibly the last level, which is filled left to right.
Level 0: A 2 = 1 node
Level 1: B C 2 = 2 nodes
Level 2: D E F G 2 = 4 nodes
Level 3: H I J K partially filled, left to right [OK]
3. Perfect Binary Tree
All internal nodes have exactly 2 children, AND all leaves are at the same level.
Every level has exactly 2 nodes (n = level number).
Level 0: A 2 = 1
Level 1: B C 2 = 2
Level 2: D E F G 2 = 4

Memory trick:
- Full = every node is 0 or 2 children
- Complete = all levels full except last, filled left right
- Perfect = all levels completely full (Full + Complete combined)

Binary Tree -- 3 Traversals
Key: Node (N), Left subtree (L), Right subtree (R)

Traversal
Order
Full name

Preorder
N L R
Node Left Right (NLR)

Inorder
L N R
Left Node Right (LNR)

Postorder
L R N
Left Right Node (LRN)

Example Tree:

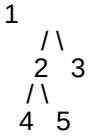
Traversal
Result
Memory trick

Preorder (NLR)
A B C = ABC
Root first

Inorder (LNR)
B A C = BAC
Root middle

Postorder (LRN)
B C A = BCA
Root last

Another Example (3 levels):



Preorder: 1, 2, 4, 5, 3
Inorder: 4, 2, 5, 1, 3
Postorder: 4, 5, 2, 3, 1

Inorder of a BST always gives sorted output -- very important!

B-Tree (Self-Balancing Tree)
A B-Tree is a self-balancing search tree in which nodes can have more than 2 children (unlike binary tree). Used in databases and file systems because it stays balanced and performs well on disk.

B-Tree is like AVL tree and Red-Black tree -- self-balancing. But B-Tree allows m children (not just 2).

Time Complexity

Operation
Complexity

Search
O(log n)

Insert
O(log n)

Delete
O(log n)

Properties of B-Tree (order m)

All leaves are at the same level
At least 2 root nodes are required (min 2 keys in root... actually min 1 key)
Each node has maximum m children
Each B-Tree node has at least m/2 children (except root and leaves)
Every node has max (m 1) keys
Min key in root = 1
Min keys in all other nodes = ** m/2   1**
Internal nodes = ** m/2 **
Maintains sorted data

Used in: Database indexes, File system directory structures (NTFS, ext4)

Graph
A graph is a non-linear data structure made up of:
- Vertices (V) -- the nodes/points
- Edges (E) -- the connections between vertices
Notation: G = (V, E)
```
    1
   / \
  2   3
 / \   \
4   5   6
```
V = {1, 2, 3, 4, 5}
E = {(1,2), (1,3), (2,3), (2,4), (2,5), (3,5), (4,5)}

Real-world uses:
- Telephone networks
- Social networks (Facebook friends)
- Circuit networks
- Road maps / GPS routing
Graph Operations

Create graph
Insert / Delete vertex
Insert / Delete edge

Graph Traversal -- BFS vs DFS
Two algorithms to visit all vertices of a graph:
BFS -- Breadth First Search

Visit all neighbours of current node first before going deeper
Uses a Queue (FIFO)
Level by level exploration

Data structures needed:
1. Visited array (track visited nodes)
2. Queue (FIFO -- for ordering exploration)
Example:
```
Graph:    1   2, 3
          2   4, 5
          3   6
BFS from 1: 1   2   3   4   5   6
```
Result: BFS | 1 | 2 | 3 | 4 | 5 | 6 |
DFS -- Depth First Search

Go as deep as possible on one path before backtracking
Uses a Stack (or recursion)
Explores one branch fully before moving to the next

Example:
DFS from 1: 1   2   4   5   3   6

BFS vs DFS -- Side by Side

Feature	BFS	DFS

Full form
Breadth First Search
Depth First Search

Strategy
Level by level
Deep then backtrack

Data structure
Queue (FIFO)
Stack (LIFO) / Recursion

Memory
More (stores all neighbours)
Less (stores path only)

Finds shortest path?
[OK] Yes (unweighted graph)
Not guaranteed

Used for
Shortest path, level-order
Cycle detection, topological sort

Trees vs Graphs -- The Difference

Feature
Tree
Graph

Cycles
No cycles
[OK] Can have cycles

Root
[OK] Has one root
No root

Edges
N nodes = N 1 edges
Any number of edges

Connected?
Always connected
May be disconnected

Special case
Tree IS a graph (acyclic, connected)
Graph is more general

Quick Cheat Sheet

```
Binary Tree       max 2 children per node
Full BT           every node has 0 or 2 children
Complete BT       all levels full except last (filled left right)
Perfect BT        all levels completely full
Traversals:
  Preorder    Root, Left, Right  (NLR)    root first
  Inorder     Left, Root, Right  (LNR)    root middle (BST   sorted!)
  Postorder   Left, Right, Root  (LRN)    root last
B-Tree        self-balancing, multi-child, O(log n) all ops, used in DB indexes
Graph         vertices + edges, can have cycles
BFS           Queue, level-by-level, finds shortest path
DFS           Stack/recursion, deep-first, used for cycle detection
```

Your 40-Second Script -- Traversals

"Binary tree has 3 traversals. Preorder visits Root then Left then Right -- root always first. Inorder visits Left then Root then Right -- for a BST this gives sorted output. Postorder visits Left then Right then Root -- root always last. Easy memory trick: Pre = root first, In = root middle, Post = root last."

Follow-Up Questions
Q: Why does Inorder of a BST give sorted output?

In a BST, left subtree has values smaller than root, right subtree has values larger. Inorder visits left first, then root, then right -- naturally produces ascending order.

Q: BFS uses Queue, DFS uses Stack -- why?

BFS explores all neighbours at current level before going deeper -- Queue's FIFO ensures we process them in the order we discovered them. DFS goes deep and backtracks -- Stack's LIFO means we always continue from the most recently discovered node.

Q: What is the difference between a Tree and a Graph?

A tree is a special case of graph -- it is connected, acyclic (no cycles), and has exactly N 1 edges for N nodes. A graph can have cycles, can be disconnected, and has no concept of a root.